

Web Development Fundamentals — Assignment Answers

Frontend, backend, client-server architecture, and web development tooling

1. Difference between frontend, backend, and full-stack development

Frontend development is everything the user sees and interacts with directly in a browser — layout, styling, buttons, forms, and animations. It is built using HTML, CSS, and JavaScript, often with frameworks such as React or Vue. Example: the product pages, cart button, and checkout screen visible on an e-commerce site like Amazon.

Backend development is the server-side logic that powers the application — handling requests, running business logic, talking to databases, and managing authentication. It is built using languages such as Node.js, Python, Java, PHP, or Ruby. Example: when a user clicks "Buy Now," the backend checks inventory, processes payment, and updates the order in the database.

Full-stack development means working on both sides — building the interface as well as the server and database logic. Example: a solo developer building a small e-commerce site from scratch, writing both the React frontend and the Node.js/Express backend.

Aspect	Frontend	Backend	Full-stack
Focus	UI/UX	Logic, data, security	Both
Languages	HTML, CSS, JS	Python, Java, Node.js, PHP	All of the above
Example role	UI developer	API/server developer	Generalist developer

2. Diagram: how the client-server model works

The client-server model works as follows: the **client** (browser or app) sends an HTTP request to the **server**; the server processes it (running logic, querying data if needed) and returns an HTTP response containing HTML, JSON, or files. Neither side initiates contact with the other except in response to a request — the client always starts the exchange.

```
[ CLIENT ] -- HTTP request --> [ SERVER ]
[ CLIENT ] <-- HTTP response -- [ SERVER ]
Sends requests, displays results      Returns HTML, JSON, or files
```

3. How a browser requests and displays a web page from a web server

- **DNS lookup** — the browser converts the domain name (e.g. example.com) into an IP address.
- **TCP/TLS connection** — the browser opens a connection to the server (HTTPS adds an encryption handshake).
- **HTTP request** — the browser sends a GET request for the page.

- **Server processing** — the server locates or generates the HTML (possibly querying a database) and sends back an HTTP response.
- **Parsing** — the browser parses the HTML into a DOM tree and the CSS into a CSSOM.
- **Rendering** — the browser combines DOM and CSSOM into a render tree, calculates layout, and paints pixels to the screen.
- **JavaScript execution** — linked scripts run and can further modify the page, such as fetching more data or adding interactivity.

4. Tools required to set up a web development environment

Tool	Purpose
Code editor (VS Code)	Writing and editing code with syntax highlighting, extensions, and debugging
Web browser (Chrome/Firefox)	Testing pages and using developer tools to inspect and debug
Version control (Git)	Tracking code changes and enabling collaboration
Git hosting (GitHub/GitLab)	Storing repositories, code review, and CI/CD
Node.js and npm	Running JavaScript outside the browser and installing packages
Package manager (npm/yarn)	Managing project dependencies
Browser DevTools	Inspecting elements, debugging JavaScript, checking network requests
Terminal / command line	Running build tools, servers, and scripts
Local/dev server (Live Server, XAMPP)	Previewing sites locally before deployment
Database client (MySQL Workbench, MongoDB Compass)	Managing and querying databases

5. What is a web server? Examples of commonly used servers

A **web server** is software (often paired with hardware) that stores website files and responds to client requests by delivering web pages, typically over HTTP/HTTPS. It listens for incoming requests, processes them, and returns content such as HTML, images, or JSON.

- **Apache HTTP Server** — widely used, highly configurable, open source.
- **Nginx** — known for speed and handling high traffic; often used as a reverse proxy or load balancer too.
- **Microsoft IIS** — used mainly in Windows-based environments.
- **Node.js (Express)** — lets developers build a custom server using JavaScript.

- **LiteSpeed** — a high-performance commercial alternative to Apache.

6. Roles: frontend developer, backend developer, and database administrator

- **Frontend developer** — builds the user interface, ensures responsiveness and accessibility, connects the UI to backend APIs, and focuses on user experience.
- **Backend developer** — writes server-side logic, builds and secures APIs, manages authentication, and connects the application to the database.
- **Database administrator (DBA)** — designs the database schema, optimizes queries and performance, manages backups, and ensures data security and integrity.

7. Install VS Code and configure it for HTML, CSS, and JavaScript development

Installing software and taking a screenshot must be done on your own computer; this is a hands-on step that cannot be performed remotely. Suggested steps:

- Download VS Code from code.visualstudio.com and install it.
- Open VS Code, go to the Extensions panel (Ctrl+Shift+X), and install: Live Server (instant HTML preview), Prettier (auto-formats code), ESLint (catches JavaScript errors), HTML CSS Support, and Auto Rename Tag.
- Create a folder containing index.html, style.css, and script.js, open it in VS Code, right-click index.html, and choose "Open with Live Server."
- Take a screenshot of the setup using your OS's screenshot tool (Windows: Win+Shift+S; Mac: Cmd+Shift+4) and include it in your submission.

8. Static vs dynamic websites

Static websites serve the same fixed HTML/CSS/JS to every visitor — the content does not change unless a developer edits the files. Example: a simple portfolio site or a company's "About Us" page.

Dynamic websites generate content on the fly, often based on user input, database queries, or session data, so different users can see different content. Example: Facebook, where each user's news feed is personalized and generated per request.

	Static	Dynamic
Content	Fixed	Changes per request/user
Speed	Very fast	Slightly slower (server processing)
Backend needed	No	Yes
Example	Personal blog (plain HTML)	Amazon, Gmail

9. Five web browsers and how their rendering engines differ

Browser	Rendering engine	Notes
Google Chrome	Blink	Google's open-source engine; fast and quick to adopt new web standards
Microsoft Edge	Blink	Switched from EdgeHTML to Chromium's Blink in 2020, so it behaves almost identically to Chrome
Mozilla Firefox	Gecko	Mozilla's own independently developed engine; strong on standards compliance
Apple Safari	WebKit	Apple's engine; also the only engine allowed for any browser on iOS
Opera / Brave / Vivaldi	Blink	Chromium-based, so they render pages the same way Chrome does

As of 2026, three engines effectively render nearly every web session: Blink, WebKit, and Gecko. Chrome, Edge, Brave, Opera, and Vivaldi all share the Blink engine, so pages look and behave almost identically across them, leaving Firefox and Safari as the only browsers with independent engines. Practically, this means developers can largely build for Blink but should still test on WebKit (Safari) and Gecko (Firefox), since each engine parses CSS, executes JavaScript, and handles edge cases slightly differently — which is why a page can look perfect in Chrome but break in Safari.

10. Labeled diagram: basic web architecture flow — client, server, database, and APIs

```
[ CLIENT ] --request--> [ SERVER ] --call--> [ API ]
[ CLIENT ] <--response-- [ SERVER ] <--data-- [ API ]
      |
      query
      v
[ DATABASE ]
```

This flow shows the typical layered architecture: the **client** sends a request to the **server**, the server either handles it directly or calls an **API** for specific business logic, and the API (or server) queries the **database** for data. The response then travels back up the same chain to the client. The database can be queried directly by the server or through the API, depending on the architecture. In modern applications, this same pattern repeats for every user action — clicking a button, loading a page, submitting a form — each triggering a fresh round trip through this cycle.